jigsaw-agile-community-rules

1st place solution

Rank	1
Team	Guanshuo Xu
Author	Guanshuo Xu
Topic ID	613305
Version	Original

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613305
Retrieved with Kaggle CLI on 2026-10-21.

Link to code is here.

I would like to thank Kaggle and Jigsaw for organizing and hosting this interesting competition. It's an excellent opportunity to explore practical aspects of LLM finetuning.

Validation strategy

What we know:

- train.csv contains two rules with comments (bodies).
- test.csv includes six rules, four of them are new.
- The four new rules have already been identified.

However, to build a strong local validation, we still need to generate realistic comments for the four new rules — a nontrivial task.

I believe the Public LB is a better choice for validation because of the following:

- The host mentioned that the public/private LB split is random, which means the public LB serves as an unbiased estimator of the private LB.
- The total test data size is not small, with the public LB representing about 30% of it. This ensures low variance in score estimation.

Therefore, all models were finetuned online and validated using the public LB.

Data processing

This notebook provides an excellent baseline for data processing and finetuning. The idea is to use positive and negative examples in the test.csv as training data. The main modifications I made to the data processing pipeline are as follows:

- Used the LLM built-in chat templates.
- I excluded the subreddit column before deduplication, since including it can create unnecessary duplicates across different subreddits. The performance improvement is significant after this change!

Finetuning

Initially, I used vanilla transformers + pytorch code, which works great, achieved 0.931 on public LB early with an ensemble of Qwen3-4b-instruct-2507, llama3.2-3b, and phi-4-mini-instruct. However, I couldn't scale beyond 7B models due to memory limitations. Later, I switched to Unsloth, which is much more memory-efficient — it can easily fit a 14B model within 16GB of GPU memory. Before finetuning, I upsampled examples from test.csv once to increase their weight, and then finetuned for one epoch. This is equivalent to one epoch for train.csv and two epochs for test.csv. Chat templates can introduce unnecessary tokens before and after the the "Yes" or "No" targets, such as "\n\n\n\n" in Qwen3 models, as well as one or more tokens following the "Yes" or "No" targets. By default, finetuning will also apply loss to these extra tokens which is unnecessary and can slow down training. To address this, I manually excluded these tokens from the loss computation, ensuring that only the "Yes" or "No" token contributes to the loss. An additional benefit of this step is that it standardizes the training behavior across different models, leading to more consistent convergence speeds and allowing me to use a single set of hyperparameters across models.

Inference

Since the last linear layer maps to the whole vocabulary, even after finetuning on the Yes/No tokens, there could still be a non-trival probabilitiy that the probs will be scattered on other tokens. So I constructed a candidate set of first tokens for many variants:

- ["Yes","YES","Y","yes","True"] -> Yes_ids
- ["No","NO","N","no","False"] -> No_ids
- plus their space-prefixed forms.

The variants are obtained by local experiments outputting the most frequent topK predicted tokens. For scoring, I output probs equivalent to the sigmoid of log-odds of Yes_ids vs No_ids.

Other tricks to speed up inference is:

- Sort test data by length
- Use forward() only. Avoids all the other operations during generation because all we need is the logits of the last token.

Postprocessing

For each model's output, I computed per-rule rankings, normalized the scores to the range [0, 1], and then ensembled based on these normalized values. This postprocessing step provided a consistent improvement, increasing the public LB score from 0.929 to 0.931 in early tests.

Model performance and final ensemble

- Larger models generally performed better.
- Qwen3 is better than the other common LLMs.
- The final ensemble is a weighted average of these six models.

model	Public LB	Private LB
Qwen3-14b	0.9297	0.9239
Qwen2.5-14b	0.9287	0.9232
Qwen3-8b	0.9272	0.9236
Qwen3-4b-instruct-2507	0.9258	0.9198
llama3.1-8b	0.9257	0.9202
Ettin-400M	0.8991	0.8944
ensemble	0.9344	0.9293

Supplemental Q&A

> **CPMP · 2025-10-25T17:48:07.210000**
>
> Thanks for the writeup and congrats.
>
> I tried ettin but found it worse than deberta v3 in my tests. Did you try deberta v3 at all?

> For each model's output, I computed per-rule rankings, normalized the scores to the range [0, 1], and then ensembled based on these normalized values. This postprocessing step provided a consistent improvement, increasing the public LB score from 0.929 to 0.931 in early tests.

I can't believe I made the mistake to rank average without grouping by rules first. Shame on me.

The models in yorur notebook are private. Is there a reason for that? If there is, then so be it. but I thought I'd ask still.

Guanshuo Xu · 2025-10-25T18:02:15.510000

I should have made all the essential models for the winning run public.
In my case both Ettin and Modernbert were better than Deberta, but I didn't try hard on them, they were only used to enrich the ensemble.

Chris Deotte · 2025-10-25T21:48:40.887000

Congratulations Guanshuo, amazing job!

During the 12 hour submit, do you finetune each of the 6 LLM models on 100% test + 100% train data for 1 epoch? That's impressive to accomplish on 2xT4 GPUs under 12 hours.

Guanshuo Xu · 2025-10-25T22:22:40.410000

Test data was upsampled, so each model finetuned on 200% test + 100% train data.

Chris Deotte · 2025-10-25T22:47:22.953000

Wow that is amazing. Unsloth must be super fast. The public notebook that trains Llama3.2-3B during submit trains with 5% test data plus 100% train data. Then infers test. It takes 6 hours and that is one LLM 3B model. And you train Qwen3-14b 0.929, Qwen2.5-14b 0.928, Qwen3-8b 0.927, Qwen3-4b-instruct-2507, llama3.1-8b, and Ettin 400M, under 12 hours with 200% test + 100% train, wow!

Do you know why your code is so much faster than this public notebook? If not, I will review your code and figure it out.

WOOSUNG YOON · 2025-10-25T23:02:39.373000

I learned a lot.
Does length-order sorting and dynamic padding significantly affect speed?
Because I used up the entire 12-hour notebook runtime to train just two models with a 3B or 4B model.
I'm curious about the key method to it.

Guanshuo Xu · 2025-10-25T23:33:54.210000

Unsloth is faster but not sure it's much faster. It takes roughly 6 hours to finetune Qwen3-14b with my code on a single T4. Spending 6 hours for finetuning a 3b model seems unreasonable …
In my method, the following all improve speed:
1) Compute loss only on a single token during training.
2) Only perform forward pass during inference, no generation.
3) Sort test data by length during inference.

Guanshuo Xu · 2025-10-25T23:39:51.863000

Does length-order sorting and dynamic padding significantly affect speed? Yes, they should.
12 hours for two 3B/4B models are indeed slow. If you scroll down my notebook, you can find some unused vanilla pytorch DDP code, I am able to train and infer two 4B models and one 3B model in roughly 3 hours with those code.

c-number · 2025-10-26T01:22:42.633000

Congrats for your 1st place!
I also used unsloth, but it almost always resulted in a lower score of about ~0.005 when compared to using transformers.
Did you experience the same issue? Or did you not have any issues with it?

Guanshuo Xu · 2025-10-26T01:44:02.357000

I don't have an issue with it. My Qwen3-4b scores using unsloth and transformers are close.

c-number · 2025-10-26T01:49:02.710000

> My Qwen3-4b scores using unsloth and transformers are close.

Did you get similar scores with the same hyperparameters (learning rate, etc.) ?

Guanshuo Xu · 2025-10-26T01:56:08.083000

Hyperparameters are mostly same, prompting are different, one use template the other does not. No matter how hard I try, the best Qwen3-4b can achieve is 0.925.

c-number · 2025-11-05T04:02:05.340000

Is there any reason why you did not use vllm for inference? @wowfattie

Guanshuo Xu · 2025-11-05T15:12:01.847000

Because I only need simple forward pass and output logits of the last token. This is more straightforward without vllm. The prompt length is short, model size not very large, and with single GPU, I don't think there is benefit using vllm, so I didn't spend time on vllm.